



ALLIANCE
UNIVERSITY

NAAC
GRADE A+
ACCREDITED UNIVERSITY

PRACTICAL MANUAL

For

ADVANCED PYTHON PROGRAMMING

(Course Code: E1CSA348)

B.Tech. for CSE

Semester – V

Academic Year: **2026-27**

Student Name :

Registration :

no.

Branch & :

Section

Faculty Name :

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU



Contents

S. No.	Title of the Practicum	Page No.
1	Zomato-like Food Delivery Management System Using Python Classes and Inheritance	
2	SBI YONO Digital Banking System Using Encapsulation, Polymorphism, and Operator Overloading	
3	Zoomcar Vehicle Rental Management System Using Class Hierarchies and Python Special Methods	
4	Amazon Customer Purchase Analytics Using Lambda, Map, Filter, and Reduce	
5	BESCOM Smart Electricity Consumption Monitoring System Using Generators and Iterators	
6	Google Fit Inspired Activity Monitoring System Using Decorators and Closures	
7	IDM-like Multi-File Download Manager Using Multithreading	
8	Zoho Payroll Processing System Using Multiprocessing and AsyncIO	
9	NPTEL Online Examination System with Error Logging and Memory Profiling	
10	Google Maps Inspired Smart Traffic Data Analytics System	
11	Naukri Candidate Registration Validation and Reporting System	
12	Apollo Hospital Patient Data Processing and Visualization System	
13	Hospital Management System Using Streamlit and SQLite	
14	Amazon Product Information Extraction System Using BeautifulSoup, Selenium, and Streamlit	
15	OpenWeather Information Dashboard Using Requests, SQLite3, and Streamlit	



Continuous Assessment Record (10 Marks)

Enter the marks obtained for each experiment (maximum 10 marks). At the end of the semester, convert the total marks (out of 150) to 10 marks.

Experiment No.	Experiment Title	Max Marks	Marks Obtained	Faculty signature
1	Food Delivery Management System	30		
2	Digital Banking System	30		
3	Vehicle Rental System	30		
4	Customer Purchase Records	30		
5	Electricity Monitoring System	30		
6	Activity Monitoring System	30		
7	Multi-File Download Manager	30		
8	Payroll Processing System	30		
9	Online Examination System	30		
10	Smart Traffic Data Analytics	30		
11	Candidate Registration Validation	30		
12	Patient Data Processing & Visualization	30		
13	Patient Management System	30		
14	Product Information Extraction	30		



15	Weather Information Dashboard	30		
----	-------------------------------------	----	--	--

Continuous Assessment Summary	Marks
Total Marks Obtained (Out of 450)	
Continuous Assessment Marks (Converted to 10)	

Faculty Signature: _____

Date: _____



Experiment No. 1

Date:

Zomato-like Food Delivery Management System Using Python Classes and Inheritance

Problem Statement

A food delivery company requires a system to manage customers, premium customers, delivery partners, restaurants, and orders.

Design and implement a Food Delivery Management System using Python classes and inheritance.

Suggested Classes

User, Customer, PremiumCustomer, DeliveryPartner, Restaurant, Order

Suggested Methods

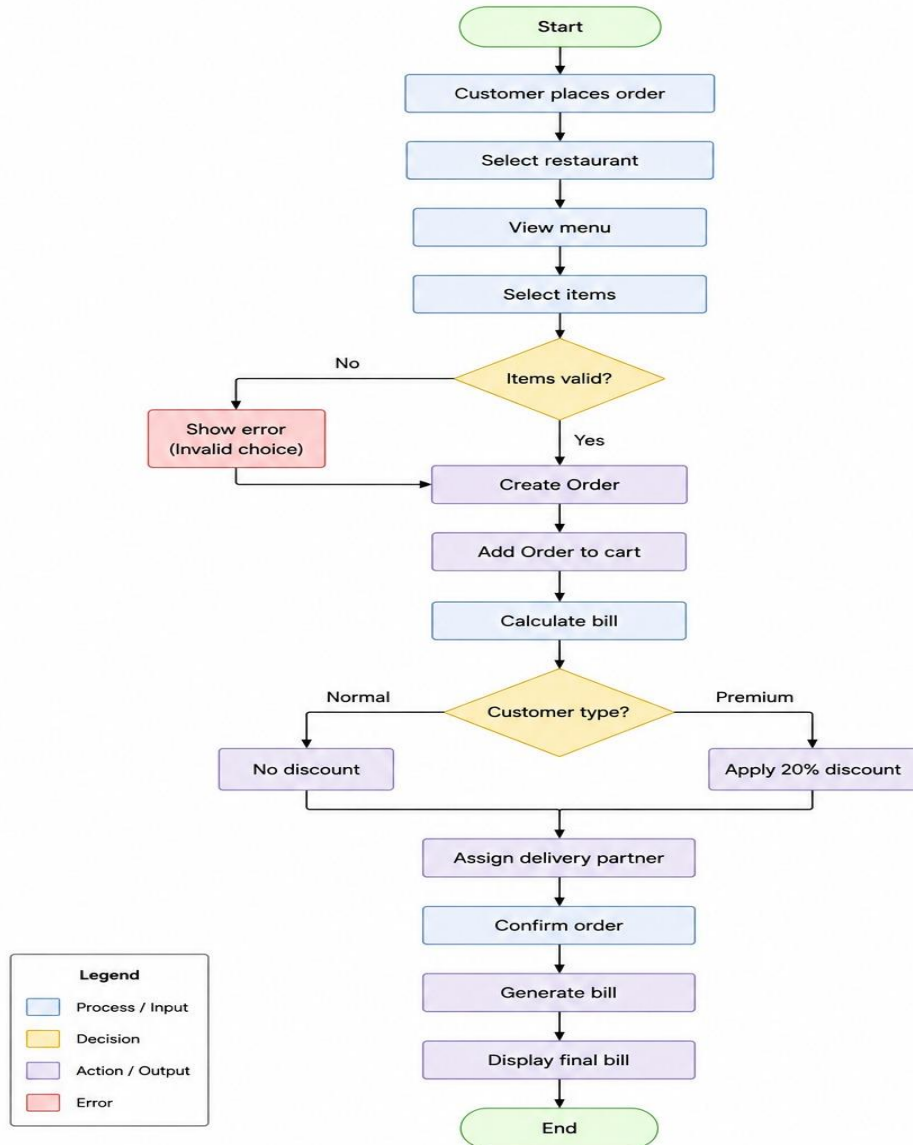
place_order(), calculate_discount(), assign_delivery(), generate_bill(), display_details()

Tools / Library Used

Python IDLE / VS Code / Jupyter Notebook

Python Features Used

Python Built-in OOP Features

Flow Chart:**Food Ordering System – Simple Process Flow**



Step-wise Implementation

Implementation Steps

- ☐ Create a class User with common attributes such as User ID and Name.
- ☐ Create a class Customer that inherits from the User class.
- ☐ Create a class PremiumCustomer that inherits from Customer.
- ☐ Create a class DeliveryPartner with delivery partner details.
- ☐ Create a class Restaurant to store restaurant information.
- ☐ Create a class Order to manage customer orders.
- ☐ Implement constructors (`__init__()`) for all classes.
- ☐ Implement `place_order()`.
- ☐ Implement `calculate_discount()`.
- ☐ Implement `assign_delivery()`.
- ☐ Implement `generate_bill()`.
- ☐ Implement `display_details()`.
- ☐ Create objects for all classes.
- ☐ Demonstrate inheritance.
- ☐ Execute and verify the output.

Challenge Task (Faculty May Assign Any One)

Modification Task

- ☐ Add multiple restaurants.
- ☐ Allow a customer to place multiple orders.
- ☐ Apply different discounts for Premium Customers.
- ☐ Display estimated delivery time.
- ☐ Generate a detailed order summary.

Viva Questions

- ☐ What is a class?
- ☐ What is an object?
- ☐ Why is inheritance used?
- ☐ Difference between Customer and PremiumCustomer?
- ☐ Which class should contain `generate_bill()`? Why?
- ☐ Explain the purpose of `__init__()`.

Code:

```
class User:

    def __init__(self, user_id, name):

        self.user_id = user_id

        self.name = name
```



```
def display_details(self):  
    print(f"User ID: {self.user_id}, Name: {self.name}\n")
```

Customer class

```
class Customer(User):
```

```
    def __init__(self, user_id, name):  
        super().__init__(user_id, name)  
        self.orders = []
```

```
    def place_order(self, restaurant, items_selected):
```

```
        for choice in items_selected:
```

```
            if 1 <= choice <= len(restaurant.menu):
```

```
                item, price = list(restaurant.menu.items())[choice - 1]
```

```
                order = Order(self, restaurant, item, price)
```

```
                self.orders.append(order)
```

```
                    print(f"{self.name} placed an order for {item} from  
{restaurant.name}\n")
```

```
            else:
```

```
                print(f"Invalid choice {choice}, skipping...\n")
```

Premium Customer

```
class PremiumCustomer(Customer):
```

```
    def calculate_discount(self, amount):
```

```
        discount = amount * 0.2 # 20% off
```




return amount - discount

Delivery Partner

class DeliveryPartner:

def __init__(self, partner_id, name):

self.partner_id = partner_id

self.name = name

def assign_delivery(self, order):

print(f"Delivery Partner {self.name} assigned to deliver {order.item}\n")

Restaurant class with numbered menu so its easy to order

class Restaurant:

def __init__(self, rest_id, name, type, menu):

self.rest_id = rest_id

self.name = name

self.type = type

self.menu = menu

def display_menu(self):

print(f"\nRestaurant: {self.name} ({self.type})")

print("Menu:")

for idx, (item, price) in enumerate(self.menu.items(), start=1):

print(f"{idx}. {item} - ₹{price}")

print("\n")



Order class

class Order:

def __init__(self, customer, restaurant, item, price):

self.customer = customer

self.restaurant = restaurant

self.item = item

self.price = price

def generate_bill(self):

if isinstance(self.customer, PremiumCustomer):

final_amount = self.customer.calculate_discount(self.price)

print(f"Bill for Premium Customer {self.customer.name}:
₹{final_amount}\n")

return final_amount

else:

print(f"Bill for Customer {self.customer.name}: ₹{self.price}\n")

return self.price

resturents list with menu

r1 = Restaurant("R101", "Green Leaf", "Veg", {

"Paneer Butter Masala": 180,

"Veg Biryani": 150,

"Dal Tadka": 120

})



```
r2 = Restaurant("R102", "Meat Lovers", "Non-Veg", {
    "Chicken Biryani": 250,
    "Mutton Curry": 300,
    "Fish Fry": 220
})

r3 = Restaurant("R103", "Mixed Flavors", "Veg & Non-Veg", {
    "Veg Burger": 100,
    "Chicken Burger": 150,
    "French Fries": 80
})

restaurants = [r1, r2, r3]

# Display resturents for user
print("\nAvailable Restaurants:")
for idx, rest in enumerate(restaurants, start=1):
    print(f"{idx}. {rest.name} ({rest.type})")
print("\n")

cust_type = input("Enter customer type (normal/premium): ").lower()
cust_name = input("Enter customer name: ")
cust_id = input("Enter customer ID: ")
print("\n")
```



```
if cust_type == "premium":
    customer = PremiumCustomer(cust_id, cust_name)
else:
    customer = Customer(cust_id, cust_name)

# Select ONE restaurant for ordering
rest_choice = int(input("Enter restaurant number to order from: "))
print("\n")

if 1 <= rest_choice <= len(restaurants):
    selected_rest = restaurants[rest_choice - 1]
    selected_rest.display_menu()

# Multiple item selection from that restaurant
items = input("Enter item numbers separated by commas: ")
items_selected = [int(x.strip()) for x in items.split(",")]
print("\n")
customer.place_order(selected_rest, items_selected)

# Assign delivery partner + calculate total bill
partner = DeliveryPartner("DP101", "Ramesh")
total_bill = 0

print("\n--- CART SUMMARY ---")
```



for order in customer.orders:

```
print(f"{order.item} from {order.restaurant.name} - ₹{order.price}")
```

```
partner.assign_delivery(order)
```

```
total_bill += order.generate_bill()
```

```
print(f"Final Total Bill: ₹{total_bill}\n")
```

else:

```
print("Invalid restaurant choice!\n")
```

Output:

Available Restaurants:

1. Green Leaf (Veg)
2. Meat Lovers (Non-Veg)
3. Mixed Flavors (Veg & Non-Veg)

Enter customer type (normal/premium): premium

Enter customer name: Deepak

Enter customer ID: 23

Enter restaurant number to order from: 2

Restaurant: Meat Lovers (Non-Veg)

Menu:

1. Chicken Biryani - ₹250
2. Mutton Curry - ₹300
3. Fish Fry - ₹220



Enter item numbers separated by commas: 1,2,1

Deepak placed an order for Chicken Biryani from Meat Lovers

Deepak placed an order for Mutton Curry from Meat Lovers

Deepak placed an order for Chicken Biryani from Meat Lovers

--- CART SUMMARY ---

Chicken Biryani from Meat Lovers - ₹250

Delivery Partner Ramesh assigned to deliver Chicken Biryani

Bill for Premium Customer Deepak: ₹200.0

Mutton Curry from Meat Lovers - ₹300

Delivery Partner Ramesh assigned to deliver Mutton Curry

Bill for Premium Customer Deepak: ₹240.0

Chicken Biryani from Meat Lovers - ₹250

Delivery Partner Ramesh assigned to deliver Chicken Biryani\

Bill for Premium Customer Deepak: ₹200.0

Final Total Bill: ₹640.0

Restaurant: Green Leaf, Location: Bengaluru, Type: Veg

Menu:

Paneer Butter Masala - ₹250

Veg Biryani - ₹200



Restaurant: Meat Lovers, Location: Bengaluru, Type: Non-Veg

Menu:

Chicken Biryani - ₹300

Mutton Curry - ₹400

Restaurant: Pizza Hut, Location: Bengaluru, Type: Veg

Menu:

Veg Pizza - ₹300

Cheese Burst Pizza - ₹500

Restaurant: BBQ Nation, Location: Bengaluru, Type: Non-Veg

Menu:

Grilled Chicken - ₹350

Fish Fry - ₹280

Order for Rahul is assigned to Arjun

Order for Priya is assigned to Arjun

Bill for Rahul: Paneer Butter Masala - ₹250

Bill for Priya: Chicken Biryani - ₹240.0 (Discount Applied)

Conclusion:

The Food Delivery Management System successfully demonstrates classes, inheritance, and object-oriented programming in Python.

It manages customers, restaurants, orders, delivery partners, discounts, and billing efficiently.

The system provides a simple and effective way to handle food ordering and delivery operations.

Faculty Signature: _____ Date: _____

Experiment No. 2**Date:****SBI YONO Digital Banking System Using Encapsulation, Polymorphism, and Operator Overloading****Problem Statement**

A bank requires a secure application to manage multiple account types and customer transactions. Develop a Digital Banking System demonstrating encapsulation, polymorphism, and operator overloading.

Suggested Classes

BankAccount, SavingsAccount, CurrentAccount, FixedDepositAccount

Suggested Methods

deposit(), withdraw(), calculate_interest(), get_balance(), set_pin(), __add __(), __gt __()

Tools / Library Used

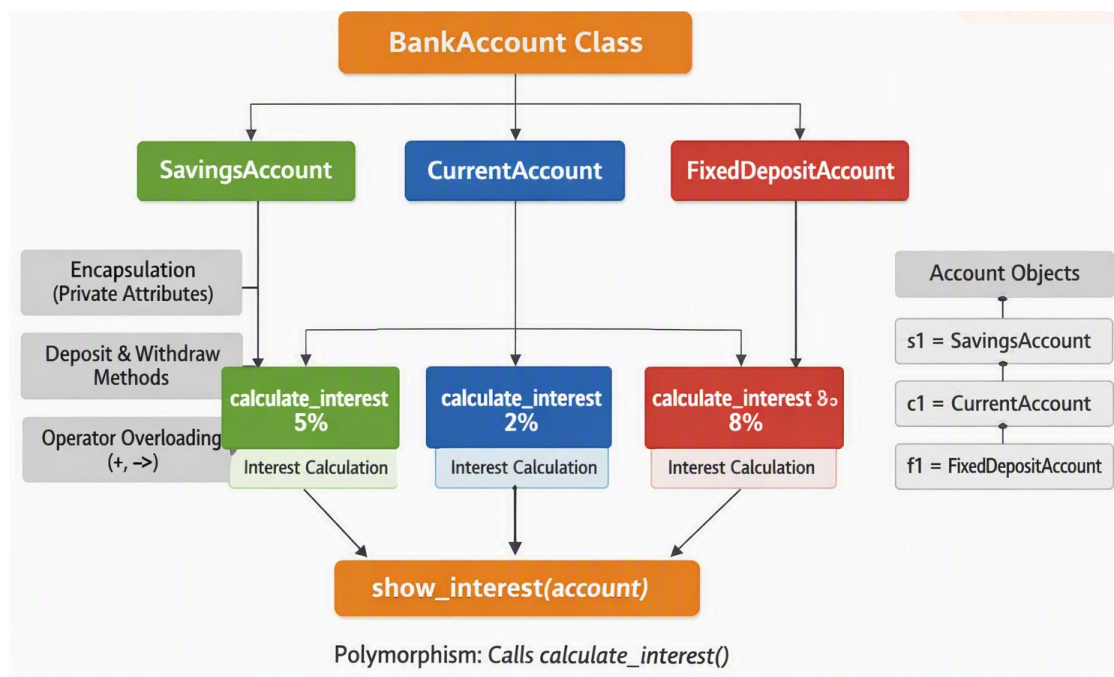
Python IDLE / Jupyter Notebook / Built-in OOP Features

Python Features Used

Encapsulation, Inheritance, Runtime Polymorphism, Operator Overloading

Constraints

- Use private attributes.
- Demonstrate runtime polymorphism.
- Overload at least two operators.

Flow chart:



Step-wise Implementation

Implementation Steps

- ☐ Create BankAccount class.
- ☐ Create SavingsAccount, CurrentAccount and FixedDepositAccount.
- ☐ Use private attributes for sensitive account information.
- ☐ Implement constructors (`__init__()`).
- ☐ Implement `deposit()`.
- ☐ Implement `withdraw()`.
- ☐ Implement `calculate_interest()`.
- ☐ Implement `get_balance()`.
- ☐ Implement `set_pin()`.
- ☐ Demonstrate runtime polymorphism.
- ☐ Overload `__add__()`.
- ☐ Overload `__gt__()`.
- ☐ Create objects for different account types.
- ☐ Execute and verify the output.

Challenge Task (Faculty May Assign Any One)

Modification Task

- ☐ Add a LoanAccount class.
- ☐ Overload one more operator.
- ☐ Implement fund transfer between accounts.
- ☐ Add PIN validation after three failed attempts.

Viva Questions

- ☐ What is encapsulation?
- ☐ Why are private attributes used?
- ☐ What is runtime polymorphism?
- ☐ Why is operator overloading used?
- ☐ What is the purpose of `__add__()`?
- ☐ What is the purpose of `__gt__()`?
- ☐ Differentiate between SavingsAccount and CurrentAccount.
- ☐ How can the banking system be improved?



Code:

```
class bankaccount:
    def __init__(self, accno, name, balance=0, pin=0000):
        self.__name = name # encapsulation private (__ double underscore then private)
        self.__accno = accno
        self.__balance = balance
        self.__pin = pin

    def get_balance(self):
        return self.__balance

    def get_name(self):
        return self.__name

    def get_accno(self):
        return self.__accno

    def set_pin(self, pin):
        if len(str(pin)) == 4:
            self.__pin = pin
            return True
        return False # Added return False for invalid pin

    def deposit(self, amount):
        self.__balance += amount
        print(f'{self.__name} deposited {amount}. New Balance: {self.__balance}')

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print(f'{self.__name} withdrew {amount}. New Balance: {self.__balance}')
            return True # Indicate successful withdrawal
```



else:

```
print("Insufficient funds!")
```

```
return False # Indicate failed withdrawal
```

Base method for interest calculation, to be overridden by subclasses

```
def calculate_interest(self):
```

```
    return 0
```

Operator Overloading methods

```
def __add__(self, other):
```

```
    if isinstance(other, bankaccount):
```

```
        return self.__balance + other.__balance
```

```
    raise TypeError("Unsupported operand type for +")
```

```
def __gt__(self, other):
```

```
    if isinstance(other, bankaccount):
```

```
        return self.__balance > other.__balance
```

```
    raise TypeError("Unsupported operand type for >")
```

```
class SavingsAccount(bankaccount):
```

```
    def __init__(self, accno, name, balance=0, pin=0000):
```

```
        super().__init__(accno, name, balance, pin)
```

```
    def calculate_interest(self):
```

```
        return self.get_balance() * 0.05
```

```
class CurrentAccount(bankaccount):
```

```
    def __init__(self, accno, name, balance=0, pin=0000):
```

```
        super().__init__(accno, name, balance, pin)
```

```
    def calculate_interest(self):
```

```
        return self.get_balance() * 0.02
```



```
class FixedDepositAccount(bankaccount):
    def __init__(self, accno, name, balance=0, pin=0000):
        super().__init__(accno, name, balance, pin)

    def calculate_interest(self):
        return self.get_balance() * 0.08

# This function should be outside the class to accept any bankaccount object
def show_interest(account):
    # Same method name, different behavior depending on object type
    print(f'Interest for {account.__class__.__name__}: {account.calculate_interest()}')

# Create objects for different account types
s1 = SavingsAccount(101, "Alice", 5000) # Corrected constructor arguments
c1 = CurrentAccount(102, "Bob", 3000)
f1 = FixedDepositAccount(103, "Charlie", 10000)

s1.deposit(2000) # Fixed typo 'deposit'
c1.withdraw(500)
f1.set_pin(1234)
show_interest(s1)
show_interest(c1)
show_interest(f1)

# Operator overloading
print("Combined Balance (s1 + c1):", s1 + c1)
print("Is s1 balance greater than c1?", s1 > c1)
```



Output:

Alice deposited 2000. New Balance: 7000

Bob withdrew 500. New Balance: 2500

Interest for SavingsAccount: 350.0

Interest for CurrentAccount: 50.0

Interest for FixedDepositAccount: 800.0

Combined Balance (s1 + c1): 9500

Is s1 balance greater than c1? True

Conclusion :

The Bank Account system successfully demonstrates encapsulation, inheritance, polymorphism, and operator overloading in Python. It efficiently manages different account types with secure PIN handling, transactions, and interest calculation.

Faculty Signature: _____ Date: _____